

Machine - Peak

1. Creds

Admin123:Admin123

spino:ButWhy123

2. Specs and Details

- name: windows-server 2019
- os: windows
- ram: 2048mb / 2GB an
- cpu: 2core
- memory: 30GB

Setup VM

- penting, make sure type nya jelas, windows 10 misal
- setup ram, memory, core, etc, jangan masukin .iso
- kelar setup, ke settings vm nya
- ke storage, ke empty bawah
- klik yang floppy disk kanan, choose disk gapapa
- then run coba vm nya

Setup windows

- di windows setup, select OS yg : Windows Server Standard (Evaluation) Desktop experience (biar ga full CLI)
- lalu pilih custom, install windows only (advanced)
- partisi: di drive full, new aja terus ok, sampai ada system dan primary, 500mb buat system
- then next aja, masukin creds
- ganti computer name nya dulu, search aja computername and rename jadi PEAK

Disable windows defender

- biar pemain bisa pakai winpeas nantinya
- ke settings search bar, ke windows security
- virus and threat protection, disable semuanya udah

3. Services

IIS - Web Server (Frontend)

Setup Awal App nya

- di server manager
- manage -> add roles and feature -> next2 in aja
- tambahkan web server IIS
- next next next in aja gausah ditambah2 plugin feature2 nya
- then restart mesinnya, lalu nyalain python server
- python -m http.server
- curl ip:8000/dikirim.zip -o app.zip

Issue dari react router

- di react router beda sama SPA single page application
- jadi doi gabisa di akses endpoint2 nya
- akses /secret misal dia gabisa
- jadi di IIS kita pakai module namanya URL-Rewrite
- <https://www.iis.net/downloads/microsoft/url-rewrite>
- downloadnya bisa disini versi x64 aja
- https://download.microsoft.com/download/1/2/8/128E2E22-C1B9-44A4-BE2A-5859ED1D4592/rewrite_amd64_en-US.msi
- then setup web.config juga untuk ini

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
  <system.webServer>
    <rewrite>
      <rules>
        <!-- Proxy /api/* ke backend -->
        <rule name="API Proxy" stopProcessing="true">
          <match url="^api/(.*)" />
          <action type="Rewrite" url="http://127.0.0.1:8080/{R:1}" />
        </rule>
        <!-- SPA Fallback -->
        <rule name="SPA Fallback" stopProcessing="true">
          <match url=".*" />
          <conditions logicalGrouping="MatchAll">
            <add input="{REQUEST_FILENAME}" matchType="IsFile" negate="true"
            />
            <add input="{REQUEST_FILENAME}" matchType="IsDirectory"
            negate="true" />
          </conditions>
          <action type="Rewrite" url="/" />
        </rule>
      </rules>
    </rewrite>
  </system.webServer>
</configuration>
```

```

        </conditions>
        <action type="Rewrite" url="/index.html" />
    </rule>
</rules>
</rewrite>
</system.webServer>
</configuration>

```

- kirim by curl aja kayak tadi
- nanti web.config taruh samping index.html tadi

Issue dari image .webp ga ke load

- solved: ganti aja source code and file img nya semua ganti ke .jpg aja

Enable proxy

- biar bisa jalan /api/login nya sesuai proxy dari frontend
- pakai ini ternyata, install dulu x64: <https://www.iis.net/downloads/microsoft/application-request-routing>
- download nya disini : <https://go.microsoft.com/fwlink/?LinkID=615136>
- kirim by curl kayak yang tadi, install aja
- ke search bar, cari aja iis manager
- ke app req route cache, lalu ke proxy kanan
- server proxy settings, enabled aja
- apply

Issue again :

- pas akses /profile/2 error kayak js nya gitu ternyata salah di vite config nya sih

```

import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

// https://vite.dev/config/
export default defineConfig({
  plugins: [react(), tailwindcss()],
  server: {
    proxy: {
      '/api' : {
        target: 'http://127.0.0.1:8080',
        changeOrigin: true,

```

```

        rewrite: (path) => path.replace(/^\/api/, '')
      }
    },
    base: './' // disini harusnya / jangan ./
  })

```

- solved with html edit aja
- di wwwroot edit aja file index.html nya pakai notepad

```

<!doctype html>

<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Bebas+Neue&family=Rajdhani:wght@400;500;600;700&display=swap" rel="stylesheet">
    <title>peak-client</title>
    <!-- disini src edit jadi gini src="/assets"-->
    <script type="module" crossorigin src="./assets/index-DhMjRDYB.js">
  </script>
    <!-- disini juga src="/assets"-->
    <link rel="stylesheet" crossorigin href="./assets/index-D6rlGPEY.css">
  </head>
  <body>
    <div id="root"></div>
  </body>
</html>

```

API Server (Backend)

Create new users for running this Services

- buat user baru buat run schedule ini
- biar kalau nanti players revshell, dia login as user ini bukan administrator
- dengan gitu dia peluang buat privesc nantinya
- di cmd buat aja

```
net user [username] [passwordnya] /add
net user spino ButWhy123 /add
runas /user:spino cmd // biar kebuat folder user spino nya
```

Setup go server nya pakai Windows Service Control

- pindahkan dulu file binary nya ke C:\app
- lalu run aja

```
sc create GoServer binPath="C:\app\peak_server.exe" start=auto
```

- ternyata di windows go di windows service gabisa gatau kenapa

```
sc query GoServer
sc delete GoServer
```

Pakai Windows task Scheduler

- search aja di search bar, task scheduler
- create task
- lalu centang: run whether user is logged or not, and centang run with highest privilege
- di tab general di, when running the task use following user acc, set ke spino, check names aja
- klik tab triggers, set begin the task: at startup
- klik tab actions, set ke file binary servernya
- lalu ok
- cek dulu dari mesin disitu

```
curl [ip]:8080/test
```

- bisa nih tapi di akses dari luar mesin gabisa, mungkin firewall nya yang halangin

Disable firewall

- ke search bar, search aja windows defender
- lalu ke firewall and network protection
- disable aja semuanya 3 3 nya public private semuanya

Issue: schedule ga run

- schedule ga run karna user bukan member SeBatchLogonRight
- solved with: win + r, ketik secpool.msc
- local policies -> user right assignment
- log on as a batch job
- set spino disitu checkname2 aja
- apply and ok

Big Issue

- backend nya jalan, tapi kayak dia gada db nya gitu, gabisa login
- even pakai real credsnya pun tetep gabisa
- jadi matikan schedule task dulu, and coba testing pakai binary aslinya
- klik 2 kali file exe nya di /app
- then test endpoint /login nya di postman
- dan bisa njir, ternyata kayaknya karna schedule task deh beda jadinya
- solved: ke schedule task tadi, lalu klik GoServer atau schedule kita tadi
- klik properties, lalu klik action, nah modif action kita
- di bagian start in (optional) itu masukin path mu: C:\app
- then restart machine nya

4. Foothold

SQLI

implement SQLI login page (known username case)

- peak_master' --
- udah ada di bagian user repository, disitu query nya disusun vuln nya disitu
- kasih hardening dikit biar gabisa inject spasi " "
- peak_master'-- atau peak_master'/**/-- juga bisa
- cek di user_service:

```
blacklist := []string{" "}
for _, v := range blacklist {
    if strings.Contains(username, v) {
        return nil, fmt.Errorf("Nice try, nt nt")
    }
}
```

Source code

- repositories/user_repository.go

```
func (r *UserRepository) FindByUsername(username, password string)
(*models.User, error) {
    kueri := fmt.Sprintf("SELECT * FROM users WHERE username = '%s' AND
password = '%s'", username, password)
    rows, err := r.Db.Query(kueri)
    if err != nil {
        return nil, err
    }
    defer rows.Close()
    var user []models.User
    for rows.Next() {
        var u models.User
        rows.Scan(&u.Id, &u.Username, &u.Password, &u.Role)
        user = append(user, u)
    }
    if len(user) != 1 {
        return nil, fmt.Errorf("Query ga ke 1 users, ini kayaknya lagi di
sql")
    }
    return &user[0], nil
}
```

- services/user_service.go

```
func (s *UserService) Login(username, password string) (*models.User, error) {
    blacklist := []string{" "}
    for _, v := range blacklist {
        if strings.Contains(username, v) {
            return nil, fmt.Errorf("Nice try, nt nt")
        }
    }
    user, err := s.Repo.FindByUsername(username, password)
    if err != nil {
        return nil, fmt.Errorf("Invalid creds")
    }
    return user, nil
}
```

- handlers/user_handler.go

```
var jwtSecret = []byte("dragonballz")
```

```

func LoginHandler(s *services.UserService) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        if r.Method != http.MethodPost {
            w.Header().Set("Content-Type", "application/json")
            w.WriteHeader(http.StatusMethodNotAllowed)
            encoder := json.NewEncoder(w)
            encoder.SetIndent("", " ")
            json.NewEncoder(w).Encode(map[string]any{
                "message": "Method not allowed",
                "status": http.StatusMethodNotAllowed,
            })
            return
        }
        var req UserLogin
        err := json.NewDecoder(r.Body).Decode(&req)
        if err != nil {
            w.Header().Set("Content-Type", "application/json")
            w.WriteHeader(http.StatusBadRequest)
            encoder := json.NewEncoder(w)
            encoder.SetIndent("", " ")
            json.NewEncoder(w).Encode(map[string]any{
                "message": "Bad request",
                "status": http.StatusBadRequest,
            })
            return
        }
        user, err := s.Login(req.Username, req.Password)
        if err != nil {
            w.Header().Set("Content-Type", "application/json")
            w.WriteHeader(http.StatusUnauthorized)
            encoder := json.NewEncoder(w)
            encoder.SetIndent("", " ")
            json.NewEncoder(w).Encode(map[string]any{
                "message": err.Error(),
                "status": http.StatusUnauthorized,
            })
            return
        }
        claims := models.Claims{
            Id:          user.Id,
            Username:    user.Username,
            Role:        user.Role,
            RegisteredClaims: jwt.RegisteredClaims{
                ExpiresAt: jwt.NewNumericDate(time.Now().Add(24 * time.Hour)),
            },
        },
    }
}

```

```

}
// setup jwt ini with algo hs256
token := jwt.NewWithClaims(jwt.SigningMethodHS256, claims)
tokenStr, err := token.SignedString(jwtSecret)
if err != nil {
    w.WriteHeader(http.StatusInternalServerError)
    return
}
http.SetCookie(w, &http.Cookie{
    Name:     "jwt_token",
    Value:    tokenStr,
    HttpOnly: true,
    Path:     "/",
    // Secure: true, -> kalo https
    SameSite: http.SameSiteLaxMode,
})
// response api nya kalau berhasil
w.Header().Set("Content-Type", "application/json")
w.WriteHeader(http.StatusOK)
encoder := json.NewEncoder(w)
encoder.SetIndent("", " ")
json.NewEncoder(w).Encode(map[string]any{
    "message": fmt.Sprintf("Login success, hello %s (%s)",
user.Username, user.Role),
    // "token": tokenStr, buat tes di postman aja, di prod mah ga
aman nanti ketahuan
    "status": http.StatusOK,
})
}
}

```

JWT Attack

implement jwt attack

- install package dari jwt nya

```
go get github.com/golang-jwt/jwt/v5
```

- nanti set secret jwt nya -> dragonballz, ada di rockyou
- nanti kalau login success dapet jwt nya misal :

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwidXNlcm5hbWUiOiJwZWFrX21hc3Rlc
iIsInJvbGUiOiJlc2VyIiwiaXhwIjoxNzcxNjI1MzA0fQ.Zcu0NdYqRywFm4pMHwZ6CCdW6W9WzFLJ
```

- nah kalau udah ketauan algo nya HS256, dia simetris cuman butuh 1 secret aja
- hashcat --identify (jwt di atas)

```
hashcat -m 16500 -a 0 hash /usr/share/wordlists/rockyou.txt
```

- pergi ke jwt.io, decode dulu lalu ke encoder, ubah payload dan sign jwt secretnya
- inject aja setelah itu, ganti cookie nya di application inspect element
- and di dashboard with privilege admin deh

Source Code

- middlewares/jwt_middleware.go

```
var jwtSecret = []byte("dragonballz")

func JwtMiddleware(next http.HandlerFunc, reqRole string) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        // pake bearer - 1
        // authHeader := r.Header.Get("Authorization") //ambil value dari
        header ini
        // if !strings.HasPrefix(authHeader, "Bearer ") { // kalau depannya ga
        ada ini lakuin :
        // w.WriteHeader(http.StatusUnauthorized)
        // return
        // }
        // tokenStr := strings.TrimPrefix(authHeader, "Bearer ") // potong
        depannya

        // pake kuki - 2
        cookie, err := r.Cookie("jwt_token")
        if err != nil {
            w.Header().Set("Content-Type", "application/json")
            w.WriteHeader(http.StatusUnauthorized)
            json.NewEncoder(w).Encode(map[string]any{
                "message": "LOGIN DULU LAH!",
                "status": http.StatusUnauthorized,
            })
            return
        }
        tokenStr := cookie.Value
        claims := &models.Claims{}
        // dia secure kalau ada ini: jwt.WithValidMethods([]string{"HS256"})
```

```

        token, err := jwt.ParseWithClaims(tokenStr, claims, func(t *jwt.Token)
(any, error) {
            return jwtSecret, nil
        })
        if err != nil || !token.Valid {
            w.Header().Set("Content-Type", "application/json")
            w.WriteHeader(http.StatusUnauthorized)
            json.NewEncoder(w).Encode(map[string]any{
                "message": "TOKEN GA VALID!",
                "status": http.StatusUnauthorized,
            })
            return
        }
        if reqRole != "" && claims.Role != reqRole {
            w.Header().Set("Content-Type", "application/json")
            w.WriteHeader(http.StatusForbidden)
            json.NewEncoder(w).Encode(map[string]any{
                "message": "Forbidden, gabo!e",
                "status": http.StatusForbidden,
            })
            return
        }
        ctx := context.WithValue(r.Context(), "claims", claims)
        next(w, r.WithContext(ctx))
    }
}

```

IDOR

Implement IDOR

- biar nanti user nya tau endpoint dimana command injection berada
- di user_repository tambahin implement dari findbyid
- and implement di user_service tambahin getbyid
- and inject new data, cek di seeds/insert_data/cmd.sql

Source Code

- services/user_service.go

```

func (s *UserService) GetById(id int) (*models.User, error) {
    user, err := s.Repo.FindById(id)
    if err != nil {
        return nil, err
    }
}

```

```

    return user, nil
}

```

- repositories/user_repository

```

func (r *UserRepository) FindById(id int) (*models.User, error) {
    kueri := fmt.Sprintf("SELECT * FROM users WHERE id = %d", id)
    row := r.Db.QueryRow(kueri)
    var u models.User
    err := row.Scan(&u.Id, &u.Username, &u.Password, &u.Role)
    if err == sql.ErrNoRows {
        return nil, fmt.Errorf("user not found")
    }
    if err != nil {
        return nil, err
    }
    return &u, nil
}

```

- handlers/admin_handler.go

```

func GetUserByIdHandler(s *services.UserService) http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        if r.Method != http.MethodGet {
            w.Header().Set("Content-Type", "application/json")
            w.WriteHeader(http.StatusMethodNotAllowed)
            json.NewEncoder(w).Encode(map[string]any{
                "message": "Method not allowed",
                "status": http.StatusMethodNotAllowed,
            })
            return
        }
        claims, ok := r.Context().Value("claims").(*models.Claims)
        if !ok {
            w.Header().Set("Content-Type", "application/json")
            w.WriteHeader(http.StatusUnauthorized)
            json.NewEncoder(w).Encode(map[string]any{
                "message": "Status Unauthorized",
                "status": http.StatusUnauthorized,
            })
            return
        }
        path := r.URL.Path
        // dipecah pecah misal "/admin/user/1" -> ["admin", "user", "1"]
    }
}

```

```

parts := strings.Split(strings.TrimSuffix(path, "/"), "/")
rawId := parts[len(parts)-1]
// conv to int
id, err := strconv.Atoi(rawId)
if err != nil || id <= 0 {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusBadRequest)
    json.NewEncoder(w).Encode(map[string]any{
        "message": "Invalid user ID",
        "status": http.StatusBadRequest,
    })
    return
}
if claims.Role != "admin" && claims.Id != id {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusForbidden)
    json.NewEncoder(w).Encode(map[string]any{
        "message": "Ngapain ngintip ngintip, pergi sana!",
        "status": http.StatusForbidden,
    })
    return
}
user, err := s.GetById(id)
if err != nil {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusNotFound)
    json.NewEncoder(w).Encode(map[string]any{
        "message": "User not found",
        "status": http.StatusNotFound,
    })
    return
}
w.Header().Set("Content-Type", "application/json")
w.WriteHeader(http.StatusOK)
json.NewEncoder(w).Encode(map[string]any{
    "id":      user.Id,
    "username": user.Username,
    "role":    user.Role,
    "status":  http.StatusOK,
})
}
}

```

Command Injection

implement command injection

- cek di admin_handler
- nanti inject payload by json gitu -> {"host": "127.0.0.1 & whoami"}

Source Code

- handlers/admin_handler.go

```
func PingHandler() http.HandlerFunc {
    return func(w http.ResponseWriter, r *http.Request) {
        if r.Method != http.MethodPost {
            w.Header().Set("Content-Type", "application/json")
            w.WriteHeader(http.StatusMethodNotAllowed)
            json.NewEncoder(w).Encode(map[string]any{
                "message": "Method not allowed",
                "status": http.StatusMethodNotAllowed,
            })
            return
        }
        var req struct {
            Host string `json:"host"`
        }
        err := json.NewDecoder(r.Body).Decode(&req)
        if err != nil || req.Host == "" {
            w.Header().Set("Content-Type", "application/json")
            w.WriteHeader(http.StatusBadRequest)
            json.NewEncoder(w).Encode(map[string]any{
                "message": "Host required!",
                "status": http.StatusBadRequest,
            })
            return
        }

        // vuln command injection, gada sanitasi input, ngeping terus ctrl + c
        out, err := exec.Command("cmd", "/C", "ping -n 1 "+req.Host).Output()
        if err != nil {
            w.Header().Set("Content-Type", "application/json")
            w.WriteHeader(http.StatusInternalServerError)
            json.NewEncoder(w).Encode(map[string]any{
                "message": "Command failed!",
                "output": err.Error(),
                "status": http.StatusInternalServerError,
            })
            return
        }
    }
}
```

```

w.Header().Set("Content-Type", "application/json")
w.WriteHeader(http.StatusOK)
encoder := json.NewEncoder(w)
encoder.SetIndent("", " ")
encoder.Encode(map[string]any{
    "message": "Ping results ini",
    "output":  string(out),
    "status":  http.StatusOK,
})
}
}

```

- cmd/app/main.go

```

func main() {
    cfg := configs.Load()
    db := configs.InitDB()
    defer db.Close()
    userRepo := &repositories.UserRepository{Db: db}
    userService := &services.UserService{Repo: userRepo}
    mux := http.NewServeMux()
    mux.HandleFunc("/test", handlers.TestingHandler)

    // cookie session etc
    mux.HandleFunc("/login", handlers.LoginHandler(userService))
    mux.HandleFunc("/logout", handlers.LogoutHandler())
    mux.HandleFunc("/me", middlewares.JwtMiddleware(handlers.MeHandler, ""))

    // set handler nya di kasih middleware dulu dan harus admin value func
    middleware nya
    mux.HandleFunc("/admin",
    middlewares.JwtMiddleware(handlers.AdminHandler(), "admin"))
    mux.HandleFunc("/admin/users/",
    middlewares.JwtMiddleware(handlers.GetUserByIdHandler(userService), ""))
    mux.HandleFunc("/admin/ping",
    middlewares.JwtMiddleware(handlers.PingHandler(), "admin"))

    log.Println("run at http://127.0.0.1:8080")
    http.ListenAndServe(cfg.HttpAddr, middlewares.CorsMiddleware(mux))
}

```

5. Privilege Escalation

SelmpersonatePrivilege

- setup awalnya
- win + r, lalu ketik aja secpol.msc
- local policies -> user right assignments
- lalu: Impersonate a client after authentication
- add si spino disitu coba
- restart and check as spino

```
runas /user:spino cmd  
whoami /priv
```

- cek ada ga selmpersonate nya
- ternyata cek nya ga gitu, beda runas sama schedule
- jadi whoami /priv nya pas di command injection aja

```
ping 127.0.1 & whoami /priv
```

Tips copas

- gatau kenapa bidirectional gabisa copas
- jadi ya pake curl aja
- mesin windows mu tulis di dump.txt lalu

```
python -m http.server
```

- di mesin target gini

```
curl ip:8000/dump.txt
```

- tinggal copy aja

Done Set Flag

- user.txt: FLAG{c2af72c547ed4b8ce60c423cdd62221f}
- root.txt: FLAG{fe2548de40276c832a32ecb786028d02}

Walkthrough

Video

<https://youtu.be/V7U28RYUHZ0>